AI ASSISTED CODING ASS – 15.1

2303A51678

B-28

Task 1 – Student Records API

Task: Use AI to build a RESTful API for managing student records.

Instructions:

• Endpoints required:

- GET /students → List all students
- POST /students → Add a new student
- PUT /students/{id} → Update student details
- DELETE /students/{id} → Delete a student record

• Use an in-memory data structure (list or dictionary) tostore records.

• Ensure API responses are in JSON format.

Expected Output:

• Working API with CRUD functionality for student records.

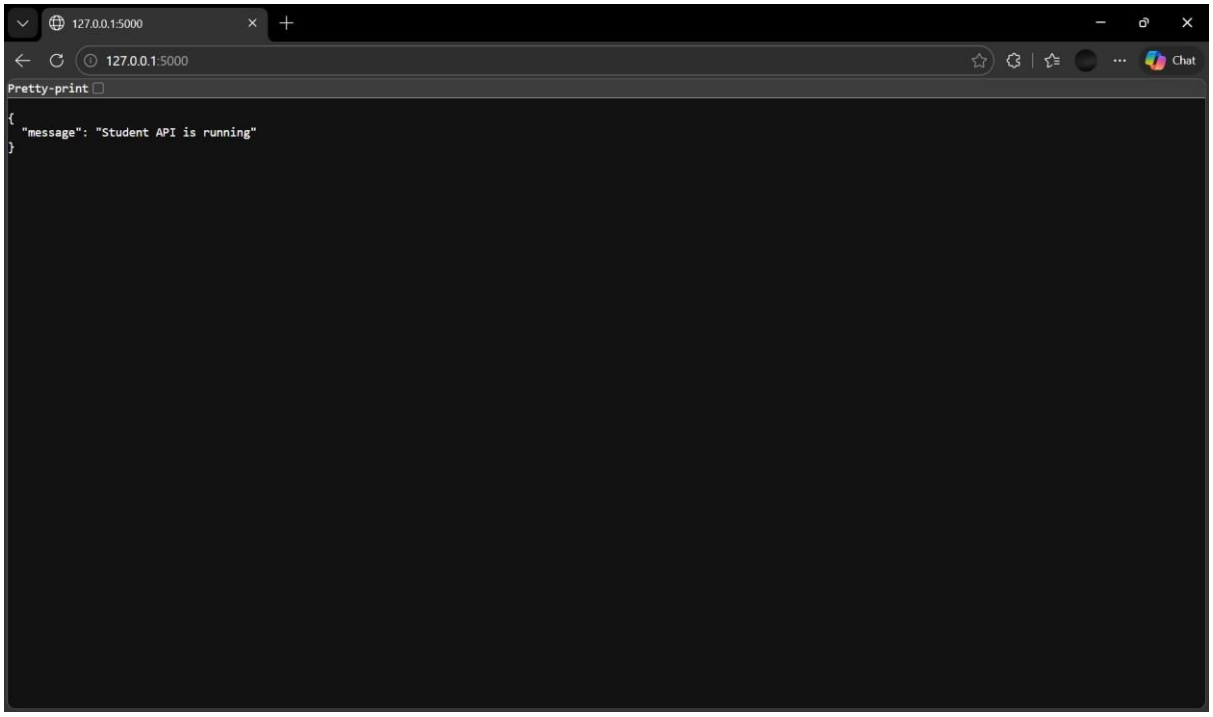
CODE:

```python
from flask import Flask, request, jsonify

app = Flask(__name__)

students = []
next_id = 1

def find_student(student_id):
    return next((s for s in students if s['id'] == student_id), None)

@app.route('/students', methods=['GET'])
def get_students():
    return jsonify({
        "count": len(students),
        "data": students
    })

@app.route('/students', methods=['POST'])
def create_student():
    global next_id
    data = request.get_json()

    if not data:
        return jsonify({"error": "No data provided"}), 400

    name = data.get("name")
    age = data.get("age")
    course = data.get("course")
```

```python
    if not name or not isinstance(age, int) or not course:
        return jsonify({"error": "Invalid input"}), 400

    student = {
        "id": next_id,
        "name": name,
        "age": age,
        "course": course
    }

    students.append(student)
    next_id += 1

    return jsonify(student), 201

@app.route('/students/<int:student_id>', methods=['PUT'])
def update_student(student_id):
    student = find_student(student_id)

    if not student:
        return jsonify({"error": "Student not found"}), 404

    data = request.get_json()

    if not data:
        return jsonify({"error": "No data provided"}), 400

    if "name" in data:
        student["name"] = data["name"]

    if "age" in data:
        if not isinstance(data["age"], int):
            return jsonify({"error": "Age must be an integer"}), 400
        student["age"] = data["age"]

    if "course" in data:
        student["course"] = data["course"]

    return jsonify(student)
```

Output:

–



Task 2 Library Book Management API

Task: Develop a RESTful API to handle library books.

Instructions:

• Endpoints required:

- GET /books → Retrieve all books
- POST /books → Add a new book
- GET /books/{id} → Get details of a specific book
- PATCH /books/{id} → Update partial book details (e.g., availability)
- DELETE /books/{id} → Remove a book

• Implement error handling for invalid requests.

Expected Output:

• Functional API with CRUD + partial updates.

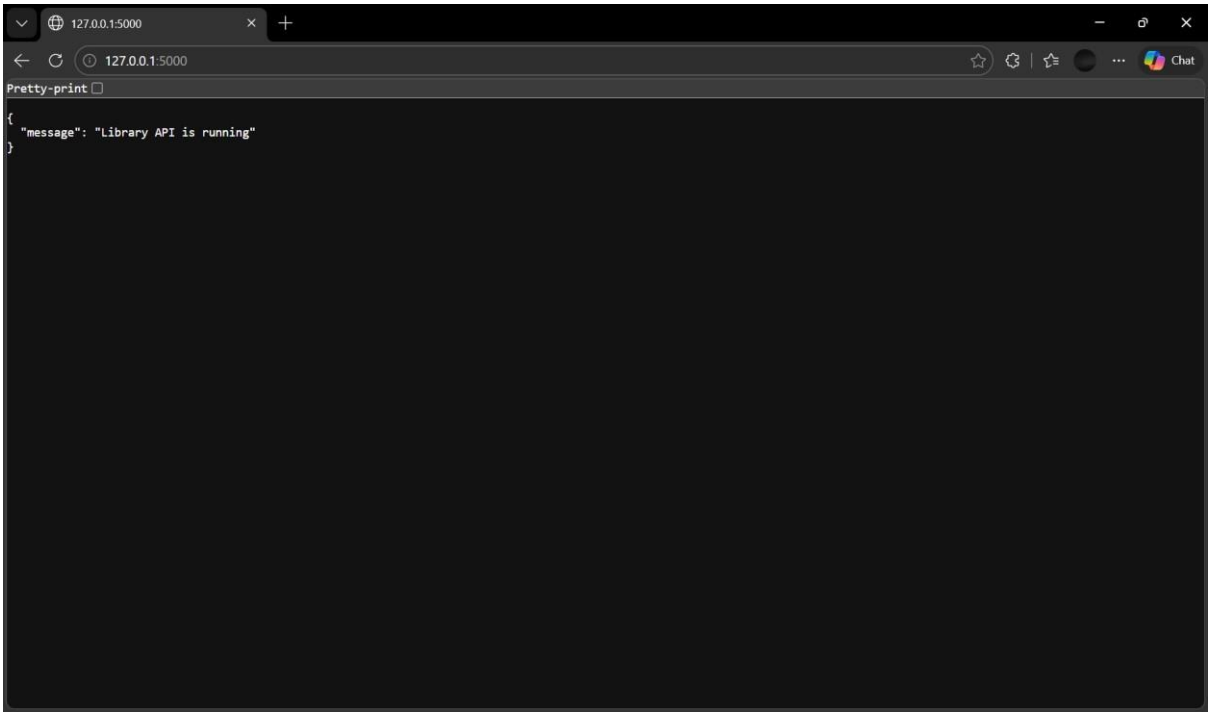
CODE:

```python
from flask import Flask, request, jsonify

app = Flask(__name__)

books = []
next_id = 1

def find_book(book_id):
    return next((b for b in books if b["id"] == book_id), None)

@app.route("/books", methods=["GET"])
def get_books():
    return jsonify(books)

@app.route("/books", methods=["POST"])
def add_book():
    global next_id
    data = request.get_json()

    if not data:
        return jsonify({"error": "No data provided"}), 400

    title = data.get("title")
    author = data.get("author")
    available = data.get("available", True)

    if not title or not author:
        return jsonify({"error": "Title and author are required"}), 400

    book = {
        "id": next_id,
        "title": title,
        "author": author,
        "available": available
    }

    books.append(book)
    next_id += 1

    return jsonify(book), 201
```

```python
@app.route("/books/<int:book_id>", methods=["GET"])
def get_book(book_id):
    book = find_book(book_id)

    if not book:
        return jsonify({"error": "Book not found"}), 404

    return jsonify(book)

@app.route("/books/<int:book_id>", methods=["PATCH"])
def update_book(book_id):
    book = find_book(book_id)

    if not book:
        return jsonify({"error": "Book not found"}), 404

    data = request.get_json()

    if not data:
        return jsonify({"error": "No data provided"}), 400

    if "title" in data:
        book["title"] = data["title"]

    if "author" in data:
        book["author"] = data["author"]

    if "available" in data:
        if not isinstance(data["available"], bool):
            return jsonify({"error": "Available must be true or false"}), 400
        book["available"] = data["available"]

    return jsonify(book)
```

Output:

–



Task 3 Employee Payroll API

Task: Create an API for managing employees and their salaries.

Instructions:

• Endpoints required:

- GET /employees → List all employees
- POST /employees → Add a new employee with salary details
- PUT /employees/{id}/salary → Update salary of an employee
- DELETE /employees/{id} → Remove employee from system

• Use AI to:

- Suggest data model structure.
- Add comments/docstrings for all endpoints.

Expected Output:

• API supporting salary management with clear documentation.

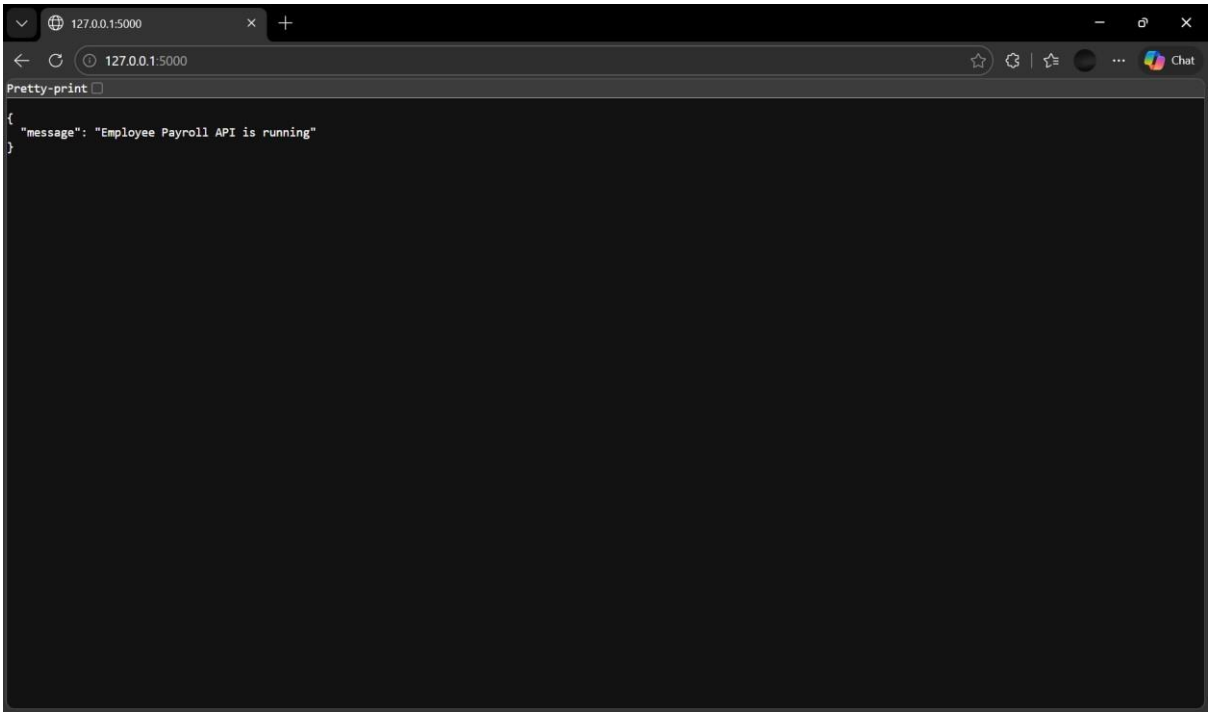
CODE:

```python
from flask import Flask, request, jsonify

app = Flask(__name__)

employees = []
next_id = 1

def find_employee(emp_id):
    return next((e for e in employees if e["id"] == emp_id), None)

@app.route("/employees", methods=["GET"])
def get_employees():
    """
    Retrieve all employees.
    Returns a list of all employee records.
    """
    return jsonify(employees)
```

–

```python
@app.route("/employees", methods=["POST"])
def add_employee():
    """
    Add a new employee with salary details.
    Expects JSON: { name, role, salary }
    """
    global next_id
    data = request.get_json()

    if not data:
        return jsonify({"error": "No data provided"}), 400

    name = data.get("name")
    role = data.get("role")
    salary = data.get("salary")

    if not name or not role or not isinstance(salary, (int, float)):
        return jsonify({"error": "Invalid input"}), 400

    employee = {
        "id": next_id,
        "name": name,
        "role": role,
        "salary": salary
    }

    employees.append(employee)
    next_id += 1

    return jsonify(employee), 201
```

Output:



Task 4 Real-Time Application: Online Food Ordering API

Scenario: Design a simple API for an online food ordering system.

Requirements:

• Endpoints required:

- GET /menu → List available dishes
- POST /order → Place a new order
- GET /order/{id} → Track order status
- PUT /order/{id} → Update an existing order (e.g., change items)
- DELETE /order/{id} → Cancel an order

• AI should generate:

- REST API code
- Suggested improvements (like authentication, pagination)

Expected Output:

Fully working API simulating a food ordering backend.

Deliverables (For All Tasks)

1. AI-generated prompts for code and test case generation.
2. At least 3 assert test cases for each task.
3. AI-generated initial code and execution screenshots.
4. Analysis of whether code passes all tests.
5. Improved final version with inline comments and explanation.
6. Compiled report (Word/PDF) with prompts, test cases, assertions, code, and output.

CODE:

```python
from flask import Flask, request, jsonify

app = Flask(__name__)

menu = [
    {"id": 1, "name": "Pizza", "price": 200},
    {"id": 2, "name": "Burger", "price": 100},
    {"id": 3, "name": "Pasta", "price": 150}
]

orders = []
next_order_id = 1

def find_order(order_id):
    return next((o for o in orders if o["id"] == order_id), None)

def calculate_total(item_ids):
    total = 0
    for item_id in item_ids:
        item = next((m for m in menu if m["id"] == item_id), None)
        if not item:
            return None
        total += item["price"]
    return total
```

```python
@app.route("/menu", methods=["GET"])
def get_menu():
    """Return all menu items"""
    return jsonify(menu)

@app.route("/order", methods=["POST"])
def create_order():
    """Create a new order"""
    global next_order_id
    data = request.get_json()

    if not data or "items" not in data or not data["items"]:
        return jsonify({"error": "Valid items required"}), 400

    total = calculate_total(data["items"])
    if total is None:
        return jsonify({"error": "Invalid item ID"}), 400

    order = {
        "id": next_order_id,
        "items": data["items"],
        "total_price": total,
        "status": "Placed"
    }

    orders.append(order)
    next_order_id += 1

    return jsonify(order), 201

@app.route("/order/<int:order_id>", methods=["GET"])
def get_order(order_id):
    """Get order details"""
    order = find_order(order_id)
    if not order:
        return jsonify({"error": "Order not found"}), 404
    return jsonify(order)
```

Output:

127.0.0.1:5000/menu

127.0.0.1:5000/menu

Pretty-print


```
[
  {
    "id": 1,
    "name": "Pizza",
    "price": 200
  },
  {
    "id": 2,
    "name": "Burger",
    "price": 100
  },
  {
    "id": 3,
    "name": "Pasta",
    "price": 150
  }
]
```